

Qwen / **Qwen3.8-27B-FP8** 

like 76

Follow  Qwen

96.8k

 Image-Text-to-Text  Transformers  Safetensors  qwen3_5  conversational  fp8

 License: apache-2.0



 Deploy ▾

→ Copy to bucket **NEW**

Use this model ▾

 **Model card**

 Files

 xet

 Community 5

Qwen3.8-27B-FP8

This repository contains FP8-quantized model weights and configuration files for the post-trained model in the Hugging Face Transformers format.

These artifacts are compatible with Hugging Face Transformers, vLLM, SGLang, TokenSpeed, etc.

The quantization method is fine-grained fp8 quantization with block size of 128, and its performance metrics are nearly identical to those of the original model.

For users seeking managed, scalable inference without infrastructure maintenance, the official Qwen API service is provided by [Qwen Cloud](#).

In particular, **Qwen3.8-27B** will be available as a hosted version with more production features, e.g., 1M context length by default, official built-in tools. For more information, please refer to the [Qwen3.8-27B Overview](#). The service is coming soon. Stay tuned for updates.

Downloads last month

 **Safetensors** 

Model size 28B params

Tensor type BF16 · F8_E4M3

 Chat template

 Files info

 **Inference Providers** **NEW**

 Image-Text-to-Text

This model isn't deployed by any Inference Provider.

 Ask for provider support

 **Model tree for Qwen/Qwen3.8-27B-...**

Base model [Qwen/Qwen3.8-27B](#)

 **Quantized** (16) [this model](#)

Quantizations    1 model

 **Collection including Qwen/Qwen3.8...**

Qwen3.8  Collection

Following the widespread community adoption of the Qwen3.5 and Qwen3.6 series, we are pleased to introduce Qwen3.8, the most capable generation in the Qwen open-model family to date.

Built on the architectural foundation of Qwen3.5, Qwen3.8 delivers substantial gains across coding, professional work, research, and long-horizon agentic tasks. Qwen3.8-27B brings these advances to a compact, deployment-friendly dense model: a native vision-language model that understands images and videos, with flexible thinking control, designed to carry complex, multi-step tasks through to completion with greater reliability.

Qwen3.8 Highlights

Qwen3.8-27B features the following enhancements:

- **Core Capabilities:** Comprehensive improvements across coding, professional work, research, and long-horizon agentic tasks.
- **Agent Execution:** Stronger autonomous planning and better handling of environment feedback, leading to more reliable end-to-end task completion.
- **Downstream Compatibility:** Broader support for popular harnesses and development tools, making it easier to integrate into your existing stack.
- **Flexible Thinking Control:** Thinking mode is on by default and can be disabled per request; reasoning depth can be tuned with `reasoning_effort`, and reasoning context from

historical messages is retained via

`preserve_thinking`.

- **Vision-Language Understanding:** Native support for image and video understanding, from STEM diagrams and documents to hour-scale videos.

Model Overview

- Type: Causal Language Model with Vision Encoder
- Training Stage: Pre-training & Post-training
- Language Model
 - Number of Parameters: 27B
 - Hidden Dimension: 5120
 - Token Embedding: 248,320 (Padded)
 - Number of Layers: 64
 - Hidden Layout: $16 \times (3 \times (\text{Gated DeltaNet} \rightarrow \text{FFN}) \rightarrow 1 \times (\text{Gated Attention} \rightarrow \text{FFN}))$
 - Gated DeltaNet:
 - Number of Linear Attention Heads: 48 for V and 16 for QK
 - Head Dimension: 128
 - Gated Attention:
 - Number of Attention Heads: 24 for Q and 4 for KV
 - Head Dimension: 256
 - Rotary Position Embedding
Dimension: 64
 - Feed Forward Network:

- Intermediate Dimension: 17,408

- LM Output: 248,320 (Padded)
- MTP (Multi-Token Prediction): trained with multiple steps
- Context Length: 262,144 natively and extensible up to 1,000,000 tokens.

Benchmark Results

Text Performance

	Qwen3.8-27B	Qwen3.6-27B	Qwen3.7-Plus
Coding			
Agentic terminal coding Terminal Bench 2.1 (Terminus)	73.0	63.4	64.0
Agentic coding SWE-bench Pro	61.7	53.5	57.6
Repo-level code generation NL2Repo-Bench	42.3	36.2	41.1
Agentic coding DeepSWE 1.1	42.2	13.3	14.2
Software engineering QwenSWEBench	79.0	49.3	59.2
Agent			
Long-horizon office work CoWorkBench	70.7	61.0	65.1
Professional job tasks	33.4	21.8	27.6

	Qwen3.8-27B	Qwen3.6-27B	Qwen3.1-72B Plus
JobBench			
Frontier agentic tasks	Pass@1 20.4	Pass@1 10.6	Pass@1 13.2
Agents' Last Exam	Score 42.9	Score 27.3	Score 33.6
General			
Instruction following	79.5	69.1	79.1
IFBench			
Scientific reasoning	89.2	87.8	90.3
GPQA Diamond			
Multidisciplinary reasoning	30.8	24.0	34.7
HLE			
Competitive coding	90.3	83.9	89.6
LiveCodeBench v6			

1. SWE-bench Pro: Except for Opus4.6 Max, which uses the officially reported score, all models are evaluated with the Claude Code harness at temp=1.0, top_p=0.95, and a 256K context window. Problematic tasks were corrected, and all baseline models were re-evaluated on the refined benchmark.
2. NL2Repo-Bench: Evaluated with the Claude Code harness. To prevent reward hacking, we disable Bash commands that attempt to access the specific repository, such as pip download, pip install, and git clone.
3. DeepSWE 1.1: Evaluated with the Claude Code harness at temp=1.0, top_p=0.95, and a 256K context window.
4. QwenSWEBench: In-house coding benchmark for evaluating models' software engineering capabilities. Evaluated with the Claude Code harness. Reporting avg@3 with an 8-hour timeout, max_tokens=32,768, temperature=1.0, and a 256K context window.
5. CoWorkBench: In-house cowork benchmark for evaluating long-horizon tasks across computer science, finance, law, medical, and other productivity domains.
6. HLE: Judged by GPT-4o.
7. The best result in each row is shown in bold.
8. Empty cells (--) indicate that results are not yet available or not applicable.

VL Performance

	Qwen3.8-27B	Qwen3.6-27B	Qwen3.7-Plus
Agentic Multimodal Intelligence			
Computer use OSWorld-Verified	84.3	63.9	73.3
Browser use WebArena-Verified	64.8	48.8	55.3
Mobile use AndroidWorld	81.9	70.3	81.0
Application recreation RecreationBench	47.1	29.8	30.2
Multimodal tool use ClawEval-MM	Pass@3 57.4	Pass@3 42.6	Pass@3 57.4
	Average 56.9	Average 50.4	Average 60.1
Multimodal software engineering SWE-MM	38.6	25.7	30.0
Visual web development Vision2Web	62.9	45.0	42.1
General Multimodal Intelligence			
Visual math problem solving MathVision	Without CI 90.0	Without CI 85.1	Without CI 90.3
	With CI 94.6		
General visual reasoning BabyVision	Without CI 65.7	Without CI 28.9	Without CI 64.7
	With CI 85.6		
Scientific chart analysis CharXiv (RQ)	Without CI 83.7	Without CI 78.4	Without CI 85.8
	With CI 90.2		

	Qwen3.8-27B	Qwen3.6-27B	Qwen3.7-Plus
Document intelligence OmniDocBench 1.5	91.1	89.4	91.4
Real-world perception RealWorldQA	85.9	84.1	86.9
Embodied intelligence ERQA	65.5	62.5	69.8

1. MathVision, BabyVision, and CharXiv (RQ): Where both settings are available, cells report “Without CI” and “With CI” separately; otherwise, only the available setting is shown. A small number of incorrect ground-truth annotations in MathVision and CharXiv (RQ) were corrected following manual verification, and all reported scores on those benchmarks were computed using the corrected annotations.
2. MathVision: Qwen3.8-27B is evaluated using the fixed prompt: “Please reason step by step, and put your final answer within `\boxed{}`.” For the remaining models, we report the higher score from two prompt variants—one with and one without the `\boxed{}` formatting requirement.
3. WebArena-Verified: Scores are computed with the official WebArena-Verified grader under the OSWorld scaffold.
4. RecreationBench: An in-house, long-horizon application-recreation benchmark designed to evaluate hybrid-agent capabilities across five platforms: desktop (Ubuntu, macOS, and Windows), mobile (Android), and the web.
5. ClawEval-MM: Scores are reported as “Pass@3 / average score.” Pass@3 is the percentage of tasks passed in at least one of three trials; the average score is the mean benchmark score across the three trials.
6. Vision2Web: Scores are averaged across the frontend, webpage, and website categories. Evaluations use the Claude Code harness and are judged by `gpt-5.4-2026-03-05`.
7. SWE-MM: Scores are evaluated on the Claude Code harness using the public dev split of SWE-bench Multimodal, with the modifications described in Appendix 8.3 of the Claude Opus 4.7 system card.
8. Empty cells (--) indicate that results are not yet available or not applicable.

Quickstart

For streamlined integration, we recommend using Qwen3.8 via APIs.

Serving Qwen3.8

Inference efficiency and throughput vary significantly across frameworks. We recommend using the latest framework versions to ensure optimal performance and compatibility. For production workloads or high-throughput scenarios, dedicated serving engines such as SGLang, vLLM, or TokenSpeed are recommended.

Qwen3.8 can be deployed with popular inference frameworks, e.g.:

- [SGLang: Qwen3.8 Cookbook](#)
- [vLLM: Qwen3.8 Recipe](#)
- [TokenSpeed: Qwen3.8 Recipe](#)

API Usage

Qwen3.8 models operate in thinking mode by default, generating thinking content signified by `<think>\n...\n</think>\n\n` before producing the final response. To disable thinking content and obtain a direct response, refer to the examples [here](#).

We recommend using the following sets of sampling parameters for generation:

- **Thinking Mode:** temperature=1.0, top_p=0.95, top_k=20, min_p=0.0, presence_penalty=0.0, repetition_penalty=1.0
- **Instruct (or non-thinking) mode:** temperature=0.7, top_p=0.80, top_k=20, min_p=0.0, presence_penalty=1.5, repetition_penalty=1.0

Please note that the support for sampling parameters varies according to inference frameworks.

Qwen3.8 comes with official support for `reasoning_effort`, which can be used to adjust reasoning depth and control cost:

- `xhigh` (default): for complex tasks demanding thorough analysis
- `medium`: balancing accuracy and speed
- `low`: efficient reasoning optimizing for speed and cost

In addition, `preserve_thinking` is enabled by default for all workloads for the best out-of-the-box experience. To disable preserved thinking, refer to the examples [here](#).

In multi-turn agentic tasks, lower reasoning effort does not always reduce overall task completion time. Although it may produce faster per-turn responses, it can also lead to insufficient analysis, more failures, and repeated retries, which may increase total latency and token consumption.

Chat Completions API

The Chat Completions API can be used with most inference frameworks, as well as [Qwen Cloud](#).

Before starting, make sure the OpenAI Python SDK is installed and the API key and the API base URL are configured, e.g.:

```
pip install -U openai
```

```
# Set the following accordingly
```

```
export OPENAI_BASE_URL='your-base-url'
export OPENAI_API_KEY='your-api-key'
```

Text-Only Input

```
from openai import OpenAI
# Configured by environment variables
client = OpenAI()

messages = [{"role": "user", "content": "Write a short story about a cat who discovers a hidden treasure."}]

completion = client.chat.completions.create(
    model="Qwen/Qwen3.8-27B-FP8",
    messages=messages,
    extra_body={
        "chat_template_kwargs": {
            "enable_thinking": True, # on by default
            "preserve_thinking": True, # on by default
        },
    },
    reasoning_effort="xhigh", # xhigh by default
    stream=True,
    stream_options={"include_usage": True},
)

reasoning_content = ""
answer_content = ""
is_answering = False
print("\n" + "=" * 20 + "Reasoning" + "=" * 20)

for chunk in completion:
    if not chunk.choices:
        print("\nUsage:")
        print(chunk.usage)
        continue

    delta = chunk.choices[0].delta

    if hasattr(delta, "reasoning_content") and delta.reasoning_content:
        if not is_answering:
```

```

        print(delta.reasoning_content, end="", flush=True)
        reasoning_content += delta.reasoning_content
    elif hasattr(delta, "reasoning") and delta.reasoning:
        if not is_answering:
            print(delta.reasoning, end="", flush=True)
            reasoning_content += delta.reasoning_content

    if hasattr(delta, "content") and delta.content:
        if not is_answering:
            print("\n" + "=" * 20 + "Answer")
            is_answering = True
        print(delta.content, end="", flush=True)
        answer_content += delta.content

messages.append({
    "role": "assistant",
    "content": answer_content,
    "reasoning_content": reasoning_content,
    "reasoning": reasoning_content,
})

```

Image Input

```

from openai import OpenAI
# Configured by environment variables
client = OpenAI()

messages = [
    {
        "role": "user",
        "content": [
            {
                "type": "image_url",
                "image_url": {
                    "url": "https://qianwen-:
                }
            },
            {
                "type": "text",
                "text": "The centres of the :
            }
        ]
    }
]

```

```

        }
    ]
}
]

chat_response = client.chat.completions.create(
    model="Qwen/Qwen3.8-27B-FP8",
    messages=messages,
)
print("Chat response:", chat_response)

```

Video Input

```

from openai import OpenAI
# Configured by environment variables
client = OpenAI()

messages = [
    {
        "role": "user",
        "content": [
            {
                "type": "video_url",
                "video_url": {
                    "url": "https://qianwen-:
                }
            },
            {
                "type": "text",
                "text": "How many porcelain :
            }
        ]
    }
]

chat_response = client.chat.completions.create(
    model="Qwen/Qwen3.8-27B",
    messages=messages,
)

```

```

# When vLLM is launched with '--media-io-kwarg=video'
# video frame sampling can be configured via
# This feature is currently supported only in vLLM
#
# By default, 'fps=2' and 'do_sample_frames=False'
# With 'do_sample_frames=True', you can customize
# chat_response = client.chat.completions.create(
#     model="Qwen/Qwen3.8-27B-FP8",
#     messages=messages,
#     extra_body={
#         "mm_processor_kwargs": {"fps": 2,
#     },
# )

```

```

print("Chat response:", chat_response)

```

Instruct (or Non-Thinking) Mode

Qwen3.8-27B will think by default before responding. You can obtain a direct response from the model without thinking by configuring the API parameters. For example,

```

from openai import OpenAI
# Configured by environment variables
client = OpenAI()

messages = [
    {
        "role": "user",
        "content": [
            {
                "type": "image_url",
                "image_url": {
                    "url": "https://qianwen-:
            }
        ],
    },
    {
        "type": "text",
        "text": "Where is this?"
    }
]

```

```

        }
    ]
}
]

chat_response = client.chat.completions.create(
    model="Qwen/Qwen3.8-27B-FP8",
    messages=messages,
    temperature=0.7,
    top_p=0.8,
    presence_penalty=1.5,
    extra_body={
        "top_k": 20,
        "chat_template_kwargs": {"enable_thinking": True},
    },
)
print("Chat response:", chat_response)

```

If you are using APIs from Qwen Cloud, in addition to changing `model`, please use `"enable_thinking": False` instead of `"chat_template_kwargs": {"enable_thinking": False}`.

Disable Preserved Thinking

By default, Qwen3.8 retains thinking blocks from all historical messages, maintaining a complete reasoning trace across the conversation. This behavior, known as preserved thinking, ensures full context continuity and is especially beneficial for agent scenarios where decision consistency and reduced redundant reasoning are critical. It also improves KV cache utilization, optimizing inference efficiency in both thinking and non-thinking modes.

If you prefer to retain only the thinking blocks from the latest user message, you can disable this

behavior by setting `preserve_thinking` to `False`:

```
from openai import OpenAI

# Configured by environment variables
client = OpenAI()
messages = [...]
chat_response = client.chat.completions.create(
    model="Qwen/Qwen3.8-27B-FP8",
    messages=messages,
    extra_body={
        "chat_template_kwargs": {"preserve_thinking": False}
    },
)
print("Chat response:", chat_response)
```

If you are using APIs from Qwen Cloud, in addition to changing `model`, please use `"preserve_thinking": False` directly instead of wrapping it in `chat_template_kwargs`.

Best Practices

To achieve optimal performance, we recommend the following settings:

1. **Sampling Parameters:** We suggest using the following sets of sampling parameters:

- Thinking Mode: `temperature=1.0`,
`top_p=0.95`, `top_k=20`, `min_p=0.0`,
`presence_penalty=0.0`,
`repetition_penalty=1.0`
- Instruct (or non-thinking) mode:
`temperature=0.7`, `top_p=0.80`, `top_k=20`,
`min_p=0.0`, `presence_penalty=1.5`,
`repetition_penalty=1.0`

For supported frameworks, you can adjust the `presence_penalty` parameter between 0 and 2 to reduce endless repetition. However, using a higher value may occasionally result in language mixing and a slight decrease in model performance.

2. Adequate Output Length: To optimize performance on agentic tasks, we recommend allocating sufficient output length to allow the model to generate detailed and comprehensive responses. For frameworks that support separate token limits for internal reasoning and final outputs, we suggest the following configuration within the 1M context length:

- Reasoning Content: Set the maximum output length to 262,144 tokens.
- Final Response: Set the maximum output length to 131,072 tokens.

These settings provide the necessary capacity for complex reasoning while ensuring ample space for high-quality final deliverables.

3. Processing Ultra-Long Texts: Qwen3.8-27B natively supports context lengths of up to 262,144 tokens. For long-horizon tasks where the total length (including both input and output) exceeds this limit, we recommend using RoPE scaling techniques to handle long texts effectively, e.g., YaRN.

YaRN is currently supported by several inference frameworks, e.g., vLLM, SGLang, and TokenSpeed. In general, there are two

approaches to enabling YaRN for supported frameworks:

- Modifying the model configuration file:

In the `config.json` file, change the `rope_parameters` fields in `text_config` to:

```
{
  "mrope_interleaved": true,
  "mrope_section": [
    11,
    11,
    10
  ],
  "rope_type": "yarn",
  "rope_theta": 100000000,
  "partial_rotary_factor": 0.25,
  "factor": 4.0,
  "original_max_position_embedding
}
```

- Passing command line arguments:

For vLLM, you can use

```
VLLM_ALLOW_LONG_MAX_MODEL_LEN=1 vllm
```

For SGLang, you can use

```
SGLANG_ALLOW_OVERWRITE_LONGER_CONTEX
```

For TokenSpeed, you can use

```
TOKENSPEED_ALLOW_OVERWRITE_LONGER_CO
```

All the notable open-source frameworks implement static YaRN, which means the scaling factor remains constant regardless of input length, **potentially impacting performance on shorter texts**. We advise modifying the `rope_parameters` configuration only when processing long contexts is required. It is also recommended to modify the `factor` as needed. For example, if the typical context length for your application is 524,288 tokens, it would be better to set `factor` as 2.0.

4. **Long Video Understanding:** To optimize inference efficiency for plain text and images, the `size` parameter in the released `video_preprocessor_config.json` is conservatively configured. It is recommended to set the `longest_edge` parameter in the `video_preprocessor_config` file to 469,762,048 (corresponding to 224k video tokens) to enable higher frame-rate sampling for hour-scale videos and thereby achieve superior performance. For example,

```
{"longest_edge": 469762048, "shortest_ed
```

Alternatively, override the default values via engine startup parameters. For implementation details, refer to: [vLLM](#) / [SGLang](#).

Citation

If you find our work helpful, feel free to give us a

```
@misc{qwen38,  
  title = {{Qwen3.8-Max}: A New Bar for Co  
  url = {https://qwen.ai/blog?id=qwen3.8},  
  author = {{Qwen Team}},  
  month = {August},  
  year = {2026}  
}
```